

Modern Cloud Computing

Lecture 13: Cloud Applications I

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- Cloud Application Development
- Cloud Workflow and Coordination
- Cloud Software

Cloud Application Development

Good Fits for Cloud Computing

Workloads matching the **arbitrarily divisible load sharing model** (partitionable into an arbitrarily large number of units processed concurrently by many instances).

Examples:

- Web services
- Database services
- Transaction-processing services
- Data processing pipelines
- Batch analytics

Cloud computing works best when the workload can be partitioned and processed independently.

Poor Fits for Cloud Computing

Problematic cases include:

- Workloads that cannot be easily partitioned
- Applications with intensive communication among instances
- Complex HPC workflows with many dependencies
- Memory-intensive or communication-intensive applications
- Applications requiring low-latency, high-bandwidth interconnects

Cloud Application Development Challenges

Application developers still need to make architecture decisions.

Major considerations:

- Compute, I/O, and network imbalance
- Distributed execution across many servers
- Data-intensive workload behavior
- Spatial and temporal locality
- Communication between threads and instances
- Instance choice and cost-performance tradeoffs

Cloud platforms can help automate placement and balancing, but they do not eliminate application-level design responsibilities.

Data Storage for Cloud Applications

Data storage strongly influences cloud application performance.

Application developers must consider:

- Where data is stored
- How data is partitioned
- Whether data is close to compute
- Storage bandwidth
- Metadata access
- Logging and post-mortem analysis

Stateless Servers

Most cloud applications use client-server request-response interaction.

A **stateless server** treats each request as an independent transaction.

Advantages:

- Simpler failure recovery
- Better scalability
- No per-client connection state
- Easier load balancing
- Clients can retry later if no response arrives

| Statelessness is a major design pattern for scalable cloud applications.

HTTP

HTTP is a request-response application protocol.

A basic web server:

- Receives an HTTP request
- Processes the request
- Returns a response
- Does not need to remember previous client interactions

However, HTTP typically uses TCP underneath.

- Useful for reliable delivery, but also exposes servers to denial-of-service risks when attackers force the server to maintain many connection attempts.

Communicating Structured Data

Distributed applications need to communicate across systems that may differ in:

- Processor architecture
- Programming language
- Data representation
- Runtime environment

From RPC to REST

RPC, ORB, CORBA

- Clients and cloud servers very often communicate by **RPC**
- An **ORB** (Object Request Broker) generalizes this: the sending ORB turns internal data structures into a byte sequence; the receiving ORB maps it back.
- **CORBA** (Common Object Request Broker Architecture) lets applications in different languages on different architectures interoperate. At its heart is the **Interface Definition Language (IDL)**, mapped to C, C++, Java, Ruby, Lisp, Python.

SOAP and WSDL

- **SOAP** (Simple Object Access Protocol): message format based on **XML**; uses TCP and more recently UDP, and can stack over other application layer protocols. SOAP is an underlying layer of **Web Services**.
- **WSDL** (Web Services Description Language): an XML grammar describing communication between endpoints: *services, types, operations, port types, bindings, port*.

REST (Representational State Transfer)

- A style of software architecture for **distributed hypermedia** systems.
- Supports client communication with **stateless servers**; platform- and language-independent; supports **data caching**; works **in the presence of firewalls**.
- Almost always uses **HTTP** for all four **CRUD** operations: **GET**, **PUT**, and **DELETE** to read, write, and delete data.

REST vs SOAP

- To retrieve an address, a REST system sends a **URL** naming the database, the individual, and the attribute; the server responds with the address. The SOAP version is **ten lines or more of XML**
- REST is easier to **build from scratch and debug**; SOAP is supported by tools that use self-documentation, e.g., WSDL, to generate connecting code.

Cloud Workflow and Coordination

Workflow in Cloud Applications

Many cloud applications require multiple interdependent tasks.

A **workflow** describes:

- What tasks must run
- What order they follow
- What conditions trigger them
- How exceptions are handled
- What resources are needed
- What state represents progress

Task Attributes in a Workflow

A task can include:

- **Name:** unique identifier
- **Description:** what the task does
- **Actions:** effects on the environment
- **Preconditions:** what must be true before execution
- **Postconditions:** what must be true after execution
- **Attributes:** resources, actors, security, reversibility
- **Exceptions:** abnormal events and handling actions

Workflow Task Types

Cloud workflows often contain both computation and control-flow elements.

Primitive task

- Cannot be decomposed further

Composite task

- Contains a subset of tasks and their execution order

Routing task

- Connects tasks
- Triggers sequential, concurrent, or iterative execution
- Includes fork and join behavior

Fork and Join Semantics

Construct	Meaning
Fork	Trigger several successor tasks
Conditional fork	Enable only successors whose conditions are true
Exclusive fork	Enable exactly one successor
Nondeterministic fork	Enable random k out of n successors
Join	Wait for predecessor tasks
k-out-of-n join	Continue after k predecessor tasks finish
Iterative join	Repeat tasks between fork and join

Workflow Lifecycle

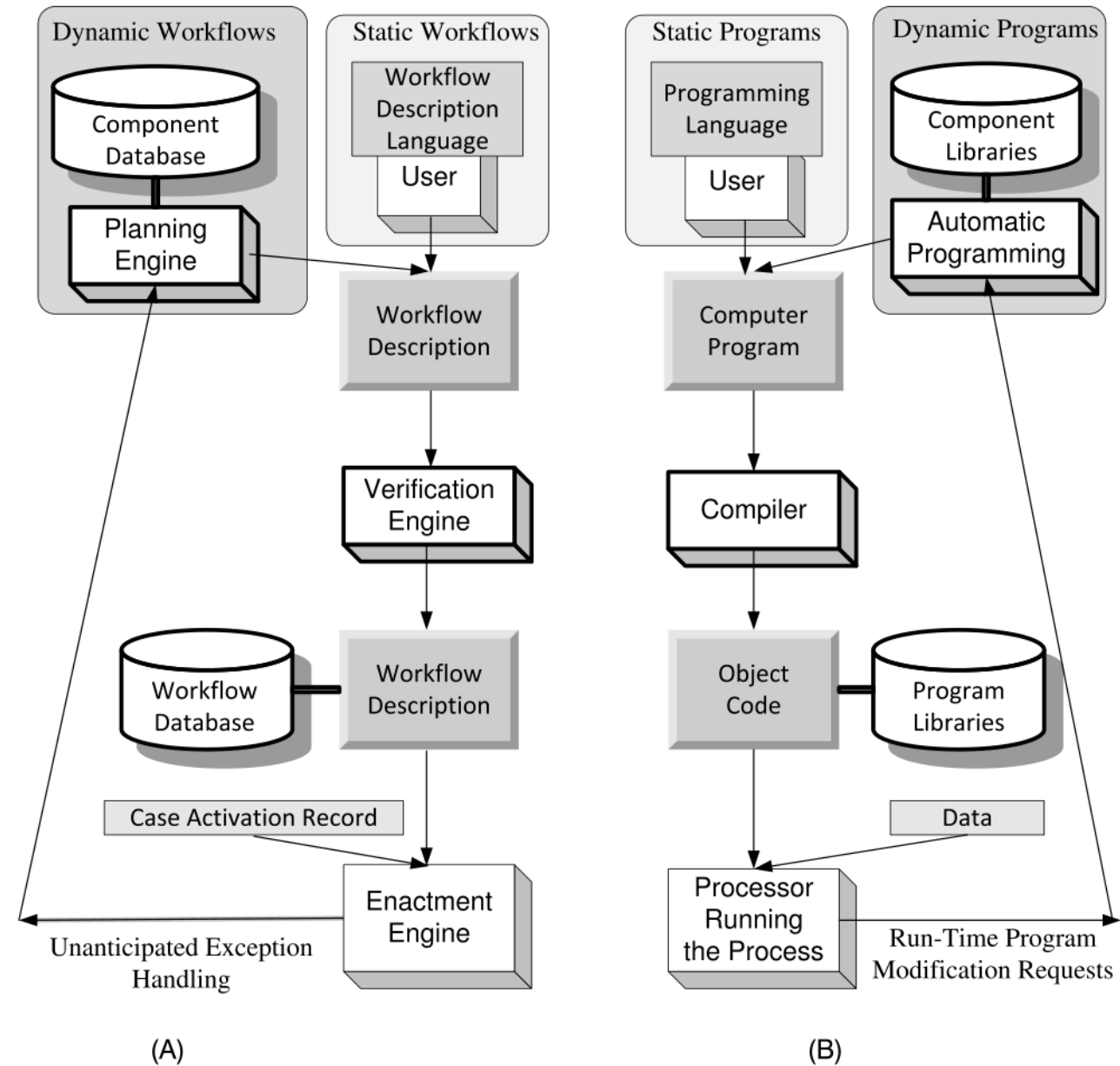
A workflow lifecycle resembles a program lifecycle.

Workflow creation

- workflow definition
- workflow verification
- workflow enactment

Program writing

- program compilation
- program execution

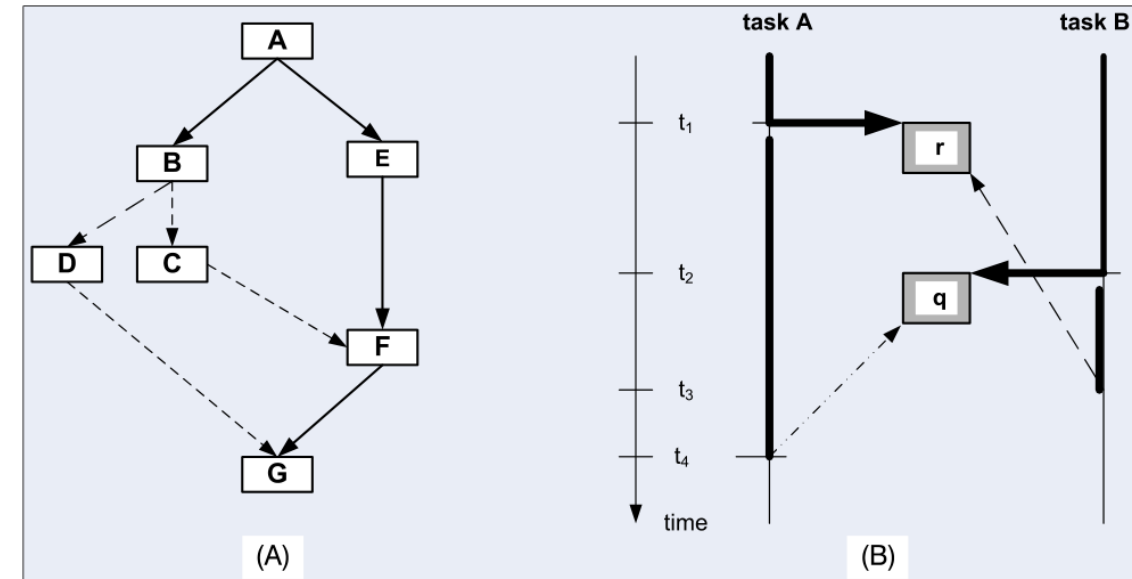


Workflow Safety and Liveness

- **Safety:** nothing bad ever happens
- **Liveness:** something good eventually happens

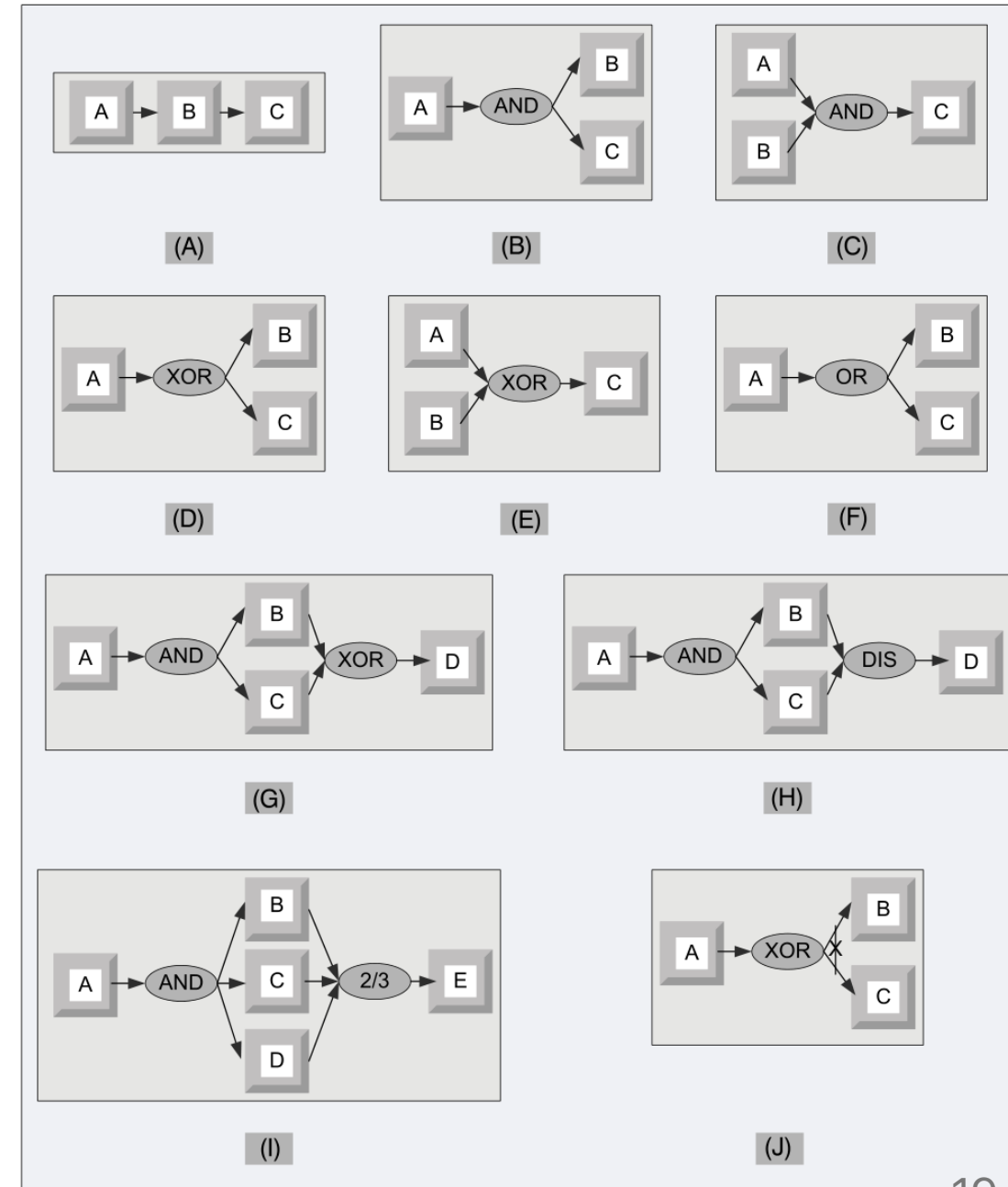
Not all processes are safe and live.

- **(A)** violates liveness. If task *C* is chosen after *B*, the process terminates after *G*. But if *D* is chosen, *F* is never instantiated (it requires both *C* and *E*) so the process never terminates, because *G* requires both *D* and *F*.
- **(B):** even a live description can **deadlock** during enactment. Tasks *A* and *B* each need exclusive access to resources *r* and *q*: at t_1 *A* acquires *r*; at t_2 *B* acquires *q*; at t_3 *B* blocks on *r*; at t_4 *A* blocks on *q*.
 - The fix: require each task to acquire **all resources at the same time**, at the price of **resource under-utilization**, since idle time per resource increases.



Workflow Patterns

- **(A) Sequence:** tasks scheduled one after another.
- **(B) AND split:** *B* and *C* both activated when *A* terminates.
Explicit: the graph has a routing node. *Implicit:* activities connect directly with conditions on branches.
- **(C) Synchronization:** *C* starts only after both *A* and *B* terminate.
- **(D) XOR split:** a decision: after *A*, either *B* or *C*.
- **(E) XOR join:** *C* is enabled when either *A* or *B* terminates.
- **(F) OR split:** after *A*, activate *B*, *C*, or **both**.
- **(G) Multiple merge:** multiple activations without synchronization; *D* runs once for *B*, again for *C*.
- **(H) Discriminator:** waits for the first incoming branch to complete, activates *D* once, then waits for the remaining branches to finish without activating *D* again, and finally resets.
- **(I) N out of M join:** barrier synchronization (any two of *A*, *B*, *C* must finish before *E* in this example).
- **(J) Deferred choice:** like XOR split, but the **run-time environment** picks the branch.



Strong vs. Weak Coordination

Strong coordination

- Coordinator supervises process group
- Acts as enactment engine
- Can support dynamic workflows
- May use hierarchical coordinators

Weak coordination

- No central supervisor
- Processes communicate peer-to-peer

ZooKeeper: Coordination Service

ZooKeeper is a distributed coordination service for large-scale systems.

It supports coordination tasks such as:

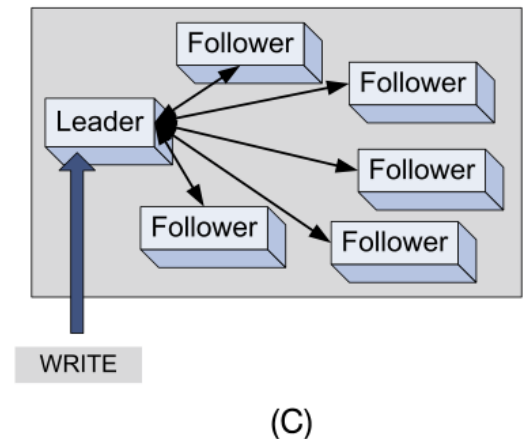
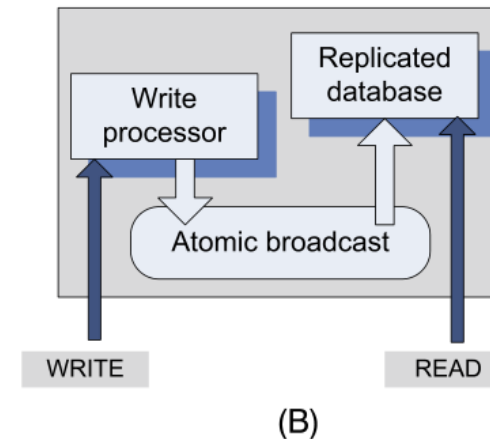
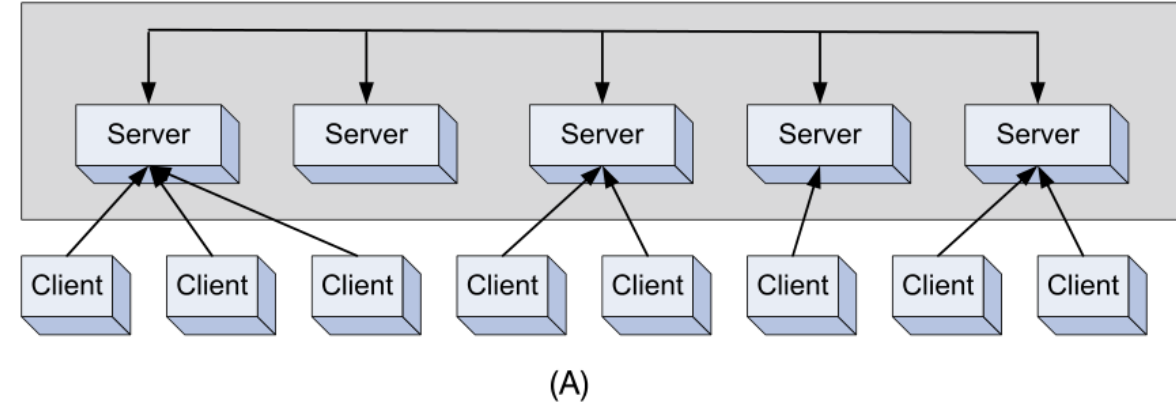
- Leader election
- Synchronization
- Recovery after failure

| ZooKeeper lets distributed components coordinate through a replicated state-machine service.

ZooKeeper Architecture

ZooKeeper runs on a pack of servers.

- Clients connect to any server
- Servers elect a leader
- Database is replicated across servers
- Reads can be served locally
- Writes go through the leader and atomic broadcast
- Service remains available while a majority of servers are available



ZooKeeper Guarantees

ZooKeeper provides several important guarantees:

- **Atomicity:** transaction either completes or fails
- **Sequential consistency:** updates applied in order
- **Single system image:** clients see the same service
- **Persistence:** updates persist until overwritten
- **Reliability:** service functions while a majority of servers remain available

Cloud Software

Current Cloud Application Categories

Existing cloud applications fall into several broad categories:

1. Processing pipelines
2. Batch processing systems
3. Web applications

Processing Pipelines

Processing pipelines are data-intensive and sometimes compute-intensive.

Examples:

- Indexing large datasets from web crawlers
- Data mining over large collections
- Image conversion, thumbnails, compression, encryption
- Video transcoding between formats
- Document conversion, encryption, and OCR

Batch Processing Systems

Batch processing is common in enterprise computing.

Examples:

- Daily, weekly, monthly, and annual reports
- Transaction aggregation for finance, insurance, and healthcare
- Inventory management
- Billing and payroll processing
- Nightly software repository updates

These applications often have deadlines, economic impact, and security requirements.

Web Applications with Variable Demand

Some web applications have periodic or temporary presence.

Examples:

- Conference websites
- Holiday-season retail services
- Tax filing services near deadlines
- Promotional campaign sites

Elasticity is valuable when demand changes dramatically over time.

Scientific Workflow Examples

- Climate and weather modeling
- Quantum chemistry calculations
- Fusion research simulation
- Accelerator modeling
- Data discovery over large scientific datasets

Volunteer Computing and Social Clouds

Volunteer computing lets users donate resources.

Examples:

- Mersenne Prime Search
- SETI@Home
- Folding@home
- Storage@Home
- BOINC-based projects

Limitations:

- Hard to use when accountability is required
- Less control over participating resources
- More difficult to provide cloud-style QoS guarantees

Summary